

Nombre: Karla Hernández Gutiérrez

Asignatura: Integración Continua en el Desarrollo Ágil

Curso: 2025-26

Máster: Desarrollo Ágil de Software para la Web

Sumario

.....	1
Enlaces del Proyecto.....	2
1. Repositorios GitHub.....	2
2. URLs de Despliegue.....	2
3. Video Explicativo.....	2
Descripción del Trabajo Realizado.....	3
Parte 1: Aplicación MovieCards Original.....	3
Se implementó la aplicación web MovieCards siguiendo la práctica del tema 5, que permite:.....	3
Parte 2: Servicio MovieCards-Service.....	3
Se creó un repositorio separado para el servicio REST basado en el código proporcionado por el profesor, con las siguientes características:.....	3
Parte 3: Integración del Servicio en la Aplicación.....	4
3.1 Modificaciones en src/main:.....	4
3.2 Modificaciones en src/test:.....	4
Parte 4: Extensión del Servicio con deadDate.....	4
Parte 5: Nueva Versión con Fecha de Fallecimiento.....	5
5.1 Workflow con Entorno Stage.....	5
5.2 Modificación del Código de la Aplicación.....	5
5.3 Modificación de Pruebas Unitarias.....	6
5.4 Modificación de Pruebas de Integración.....	6
5.5 Modificación de Pruebas End-to-End.....	6
5.6 Garantía de Calidad con SonarQube.....	7
Configuración del Pipeline CI/CD.....	7
Estructura del Workflow:.....	7
Características del Pipeline:.....	7
Tecnologías Utilizadas.....	8
Backend:.....	8
Frontend:.....	8
Infraestructura:.....	8
Base de Datos:.....	8
Gestión de Versiones y Control de Código.....	8
Arquitectura de Despliegue en Azure.....	10
Desafíos Encontrados y Soluciones.....	10
Desafío 1: Migración a Servicio Externo.....	10
Desafío 2: Pruebas con RestTemplate.....	10
Desafío 3: Pipeline Complejo.....	10
Desafío 4: Calidad del Código.....	10

Enlaces del Proyecto

1. Repositorios GitHub

- **Aplicación moviecards:** <https://github.com/KHGUAH/moviecards>
- **Servicio moviecards-service:** <https://github.com/KHGUAH/moviecards-service>

2. URLs de Despliegue

- **Aplicación web (producción):** <https://moviecards-hernandez.azurewebsites.net/>
- **Aplicación web (pre-producción):** <https://moviecards-pre-hernandez.azurewebsites.net/>
- **Servicio REST:** <https://moviecards-service-hernandez.azurewebsites.net>

Nota: Para acceder a los datos del servicio REST desde un navegador, se debe utilizar la URL completa incluyendo el endpoint específico. Por ejemplo, para obtener la lista de actores, se debe navegar a: <https://moviecards-service-hernandez.azurewebsites.net/actors>

3. Video Explicativo

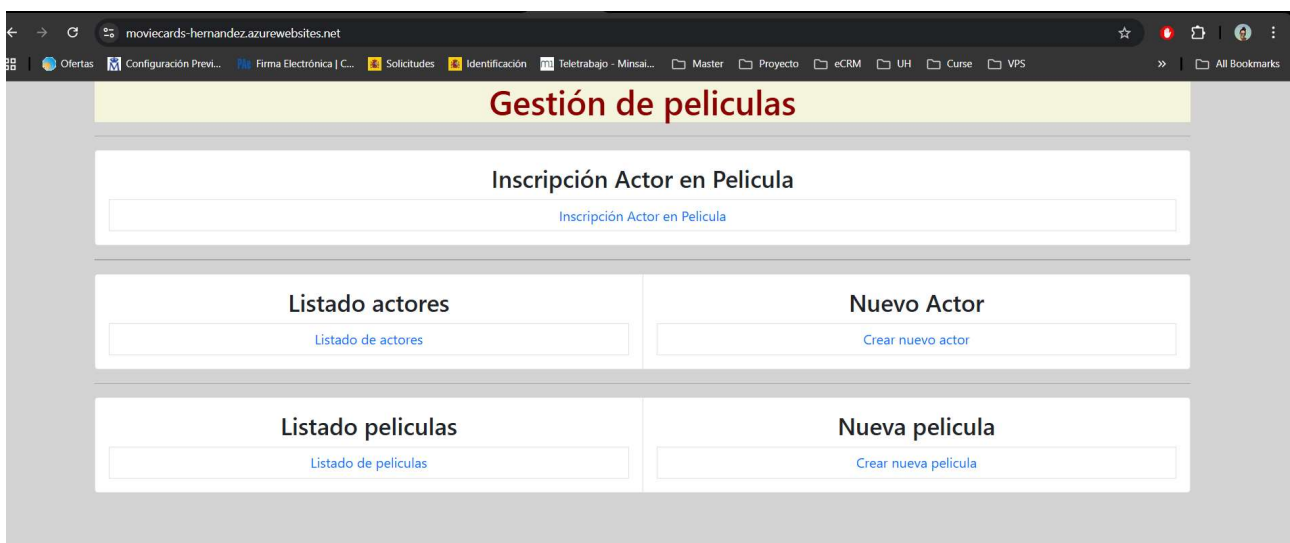
- URL del video: [ExposicionIntegracionContinua.avi](#)

Descripción del Trabajo Realizado

Parte 1: Aplicación MovieCards Original

Se implementó la aplicación web MovieCards siguiendo la práctica del tema 5, que permite:

- Gestión de actores (crear, editar, listar)
- Gestión de películas (crear, editar, listar)
- Registro de actores en películas (creación de fichas)
- Interfaz web con Thymeleaf y Bootstrap



Parte 2: Servicio MovieCards-Service

Se creó un repositorio separado para el servicio REST basado en el código proporcionado por el profesor, con las siguientes características:

1. API RESTful para gestión de actores y películas

Endpoints:

- GET /actors - Listar actores
- POST /actors - Crear actor
- GET /actors/{id} - Obtener actor por ID
- PUT /actors - Actualizar actor
- GET /movies - Listar películas
- POST /movies - Crear película
- GET /movies/{id} - Obtener película por ID
- PUT /movies - Actualizar película
- GET /movies/insc/{idActor}/{idMovie} - Inscribir actor en película

```
moviecards-service-herandez.azurewebsites.net/actors
Pretty-print
{
  "id": 1,
  "name": "Alina Rodríguez",
  "birthDate": "1951-10-04T00:00:00+00:00",
  "deadDate": "2015-07-27T00:00:00+00:00",
  "country": "Cuba",
  "movies": [
    {
      "id": 2,
      "title": "Conducta",
      "releaseYear": 2014,
      "duration": 135,
      "country": "Cuba",
      "direction": "Ernesto Daranas",
      "genre": "Drama",
      "sinopsis": "Chala desarrolla un fuerte vínculo con su maestra, y luchan juntos contra el sistema cuando su nueva profesora quiere enviarlo a un centro de reeducación."
    }
  ]
}
```

Parte 3: Integración del Servicio en la Aplicación

3.1 Modificaciones en src/main:

- **MovieCardsApplication.java:** Se añadió el bean RestTemplate para consumir servicios REST
- **ActorServiceImpl.java:** Se reemplazó ActorJPA por RestTemplate para consumir el servicio externo
- **MovieServiceImpl.java:** Se reemplazó MovieJPA por RestTemplate
- **CardController.java:** Se modificó la inyección de dependencias para usar @Autowired

3.2 Modificaciones en src/test:

- **ActorServiceImplTest.java:** Se adaptaron las pruebas unitarias para mockear RestTemplate
- **MovieServiceImplTest.java:** Se actualizaron las pruebas para el nuevo servicio REST

Identificador	Nombre	Fecha Nacimiento	Fecha Fallecimiento	Pais	Editar
1	Alina Rodríguez	04-10-1951	27-07-2015	Cuba	Editar

[Volver a la Página de Inicio](#)

Identificador	Título	Año	Duración	País	Dirección	Género	Sinopsis	Editar
2	Conducta	2014	135	Cuba	Ernesto Daranas	Drama	Chala desarrolla un fuerte vínculo con su maestra, y luchan juntos contra el sistema cuando su nueva profesora quiere enviarlo a un centro de reeducación.	Editar

[Volver a la Página de Inicio](#)

Parte 4: Extensión del Servicio con deadDate

Se extendió el servicio moviecards-service añadiendo:

- Campo deadDate (Date) en la entidad Actor
- Modificaciones en el controlador para aceptar y devolver este campo

```
moviecards-service-herandez.azurewebsites.net/actors
[
  {
    "id": 1,
    "name": "Alina Rodríguez",
    "birthDate": "1951-10-04T00:00:00.000+00:00",
    "deadDate": "2015-07-27T00:00:00.000+00:00",
    "country": "Cuba",
    "movies": [
      {
        "id": 2,
        "title": "Conducta",
        "releaseYear": 2014,
        "duration": 135,
        "country": "Cuba",
        "director": "Ernesto Daranas",
        "genre": "Drama",
        "sinopsis": "Chala desarrolla un fuerte vínculo con su maestra, y luchan juntos contra el sistema cuando su nueva profesora quiere enviarlo a un centro de reeducación."
      }
    ]
  }
]
```

Parte 5: Nueva Versión con Fecha de Fallecimiento

5.1 Workflow con Entorno Stage

Se añadió un nuevo job "stage" en el workflow de GitHub Actions que despliega a un entorno de pre-producción (moviecards-pre-herandez) antes del despliegue a producción.

```
stage:
  needs: qa
  runs-on: ubuntu-latest
  steps:
    - name: Descargar artefacto del build
      uses: actions/download-artifact@v4
      with:
        name: moviecards-java
    - name: Deploy to Azure Web App (Pre-Producción)
      uses: azure/webapps-deploy@v3
      with:
        app-name: 'moviecards-pre-herandez'
        publish-profile: '${{ secrets.AZUREAPPSERVICE_PUBLISHPROFILE_25AE17465ABC44659AA714B720622419 }}'
        package: '*.jar'
```

5.2 Modificación del Código de la Aplicación

- Se añadió el campo deadDate en el formulario de actores
- Se mostró la fecha de fallecimiento en el listado de actores
- Se implementó la lógica para manejar fechas opcionales de fallecimiento

moviecards-herandez.azurewebsites.net/actors

Identificador	Nombre	Fecha Nacimiento	Fecha Fallecimiento	Pais	Editar
1	Alina Rodríguez	04-10-1951	27-07-2015	Cuba	<button>Editar</button>

Volver a la Página de Inicio

moviecards-herandez.azurewebsites.net/actors/new

Nuevo Actor

Nombre
Escriba el nombre del actor

Fecha nacimiento
mm/dd/yyyy

Fecha fallecimiento
mm/dd/yyyy

Pais
Escriba el país del actor

Guardar

Volver a la Página de Inicio

5.3 Modificación de Pruebas Unitarias

- **ActorTest.java:** Se añadieron pruebas para `getDeadDate()` y `setDeadDate()`
- **ActorServiceImplTest.java:** Se actualizaron para incluir `deadDate` en los objetos de prueba

```
@Test
void testSetGetDeadDate() {
    Date deadDateExample = new Date();
    actor.setDeadDate(deadDateExample);
    assertEquals(deadDateExample, actor.getDeadDate());
}
```

5.4 Modificación de Pruebas de Integración

- **ActorJPAIT.java:** Se actualizaron las pruebas de integración para incluir `deadDate`
- Se verificó la persistencia correcta del campo `deadDate` en la base de datos

```
@Test
public void testSaveActor() {
    Actor actor = new Actor();
    actor.setName(name: "actor");
    actor.setBirthDate(new Date());
    actor.setDeadDate(new Date());
    actor.setCountry(country: "spain");

    Actor savedActor = actorJPA.save(actor);

    assertNotNull(savedActor.getId());

    assertEquals(actor.getDeadDate(), savedActor.getDeadDate());

    Optional<Actor> foundActor = actorJPA.findById(savedActor.getId());

    assertTrue(foundActor.isPresent());
    assertEquals(savedActor, foundActor.get());
}
```

```
@Test
public void testFindById() {
    Actor actor = new Actor();
    actor.setName(name: "actor");
    actor.setBirthDate(new Date());
    actor.setDeadDate(new Date());
    Actor savedActor = actorJPA.save(actor);

    Optional<Actor> foundActor = actorJPA.findById(savedActor.getId());

    assertTrue(foundActor.isPresent());
    assertEquals(savedActor, foundActor.get());
    assertEquals(savedActor.getDeadDate(), foundActor.get().getDeadDate());
}
```

5.5 Modificación de Pruebas End-to-End

Se actualizaron las pruebas funcionales para:

- Verificar que el campo `deadDate` aparece en el formulario
- Comprobar que se muestra correctamente en el listado
- Validar el comportamiento con fechas de fallecimiento nulas

```
@Test
public void testListActors() {
    driver.get(url: "http://localhost:8089/actors");
    WebElement title = driver.findElement(By.className(className: "card-header"));
    assertEquals(expected: "Listado Actores", title.getText());

    WebElement table = driver.findElement(By.className(className: "table-hover"));

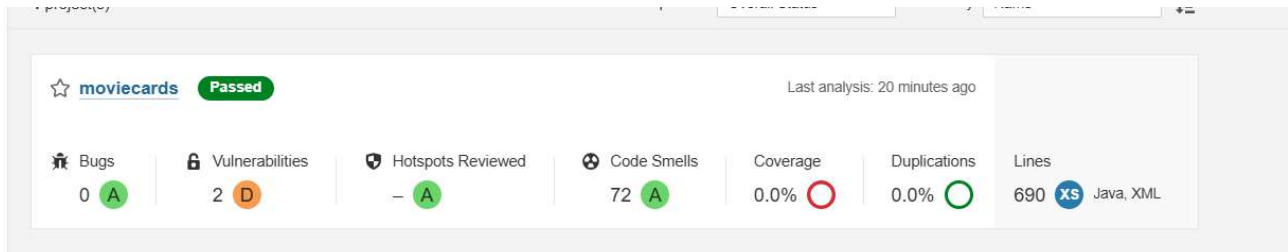
    WebElement thead = table.findElement(By.tagName(tagName: "thead"));
    assertTrue(thead.isDisplayed());

    WebElement headerRow = thead.findElement(By.tagName(tagName: "tr"));
    assertEquals(expected: "Identificación", headerRow.findElement(By.tagName(tagName: "th")).getText());
    assertEquals(expected: "Nombre", headerRow.findElement(By.tagName(tagName: "th")).getText());
    assertEquals(expected: "Fecha Nacimiento", headerRow.findElement(By.tagName(tagName: "th")).getText());
    assertEquals(expected: "Fecha Fallecimiento", headerRow.findElement(By.tagName(tagName: "th")).getText());
    assertEquals(expected: "País", headerRow.findElement(By.tagName(tagName: "th")).getText());
    assertEquals(expected: "Editor", headerRow.findElement(By.tagName(tagName: "th")).getText());
}
```

5.6 Garantía de Calidad con SonarQube

Configuración del pipeline:

- El job `qa` ejecuta SonarQube en cada commit
- Si SonarQube detecta 5 o más problemas críticos, el pipeline FALLA Solo se permite el despliegue a producción si hay menos de 5 problemas críticos



Configuración del Pipeline CI/CD

Estructura del Workflow:

- **build:** Compilación del proyecto con Maven
- **test:** Ejecución de pruebas unitarias, de integración y end-to-end
- **qa:** Análisis de calidad con SonarQube
- **stage:** Despliegue a entorno de pre-producción
- **deploy:** Despliegue a producción (requiere aprobación manual)



Características del Pipeline:

- Ejecución automática en cada push
- Pruebas automatizadas en cada commit
- Análisis de calidad con SonarQube
- Despliegue en dos entornos (stage y production)
- Aprobación manual para producción
- Validación de calidad (<5 problemas críticos)

Tecnologías Utilizadas

Backend:

- **Java 11**
- **Spring Boot**
- **Spring MVC**
- **Spring Data JPA**
- **Hibernate**
- **JUnit 5**
- **Mockito**

Frontend:

- **Thymeleaf**
- **HTML5/CSS3**
- **Bootstrap 5**

Infraestructura:

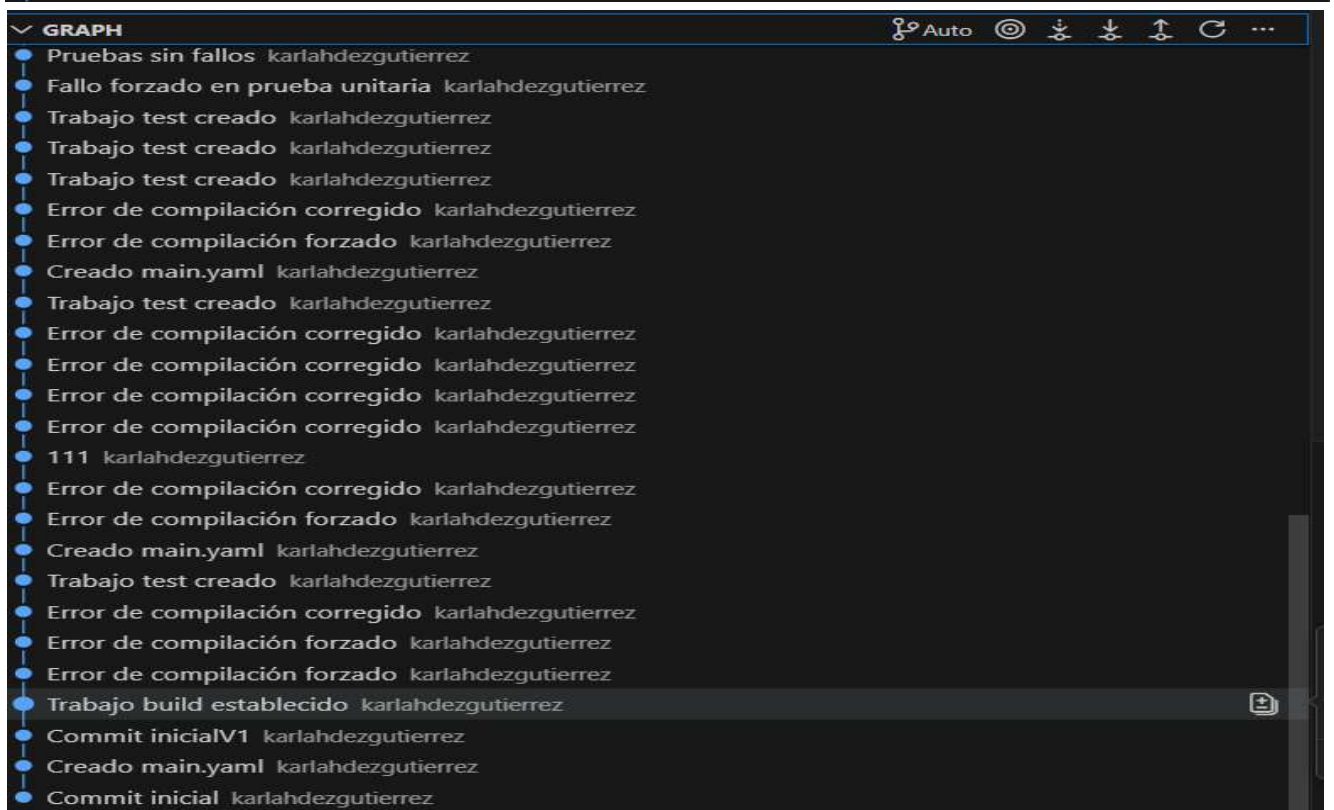
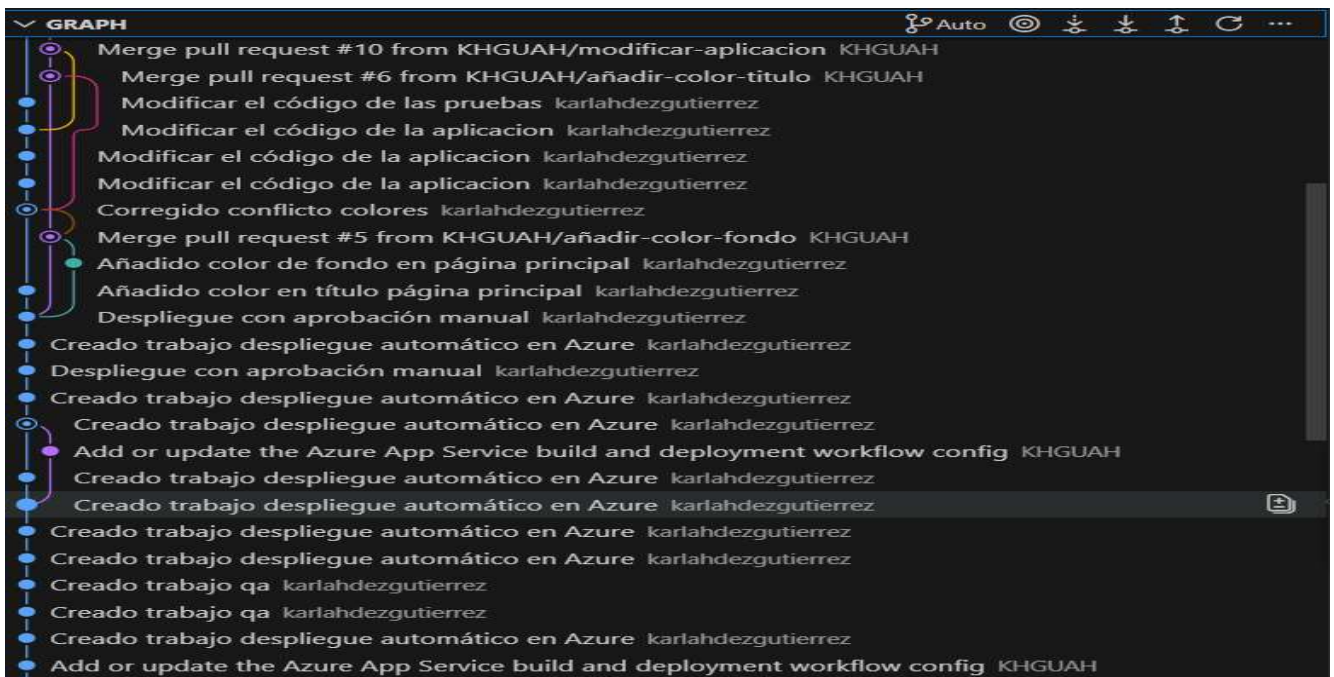
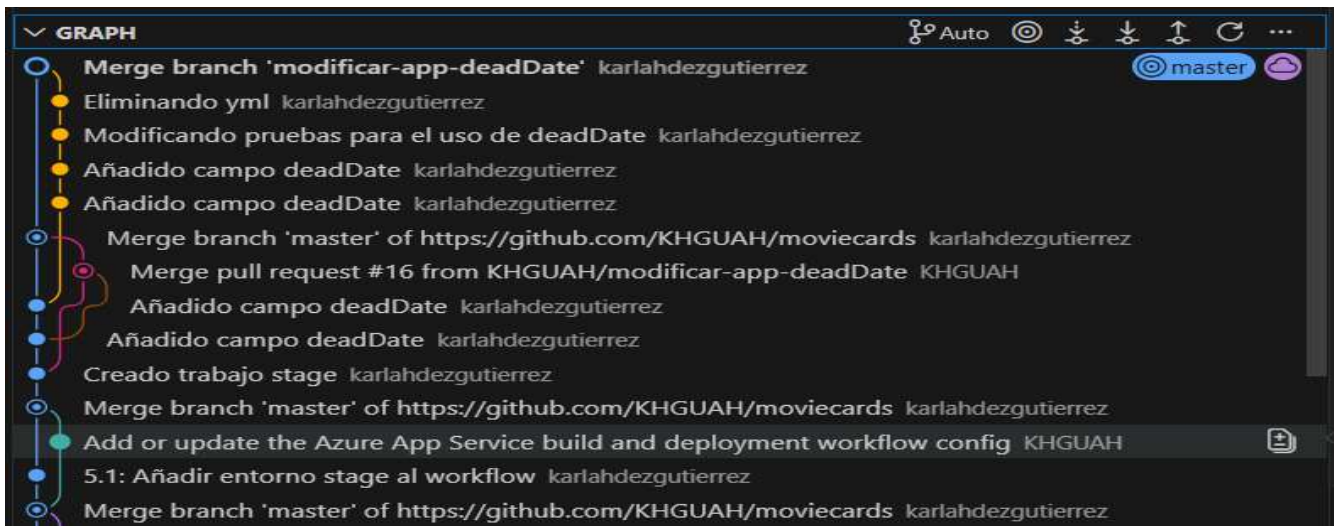
- **Azure App Service**
- **GitHub Actions**
- **SonarQube**
- **Maven**

Base de Datos:

- **H2 Database** (desarrollo/pruebas)
- **Azure Database** (producción)

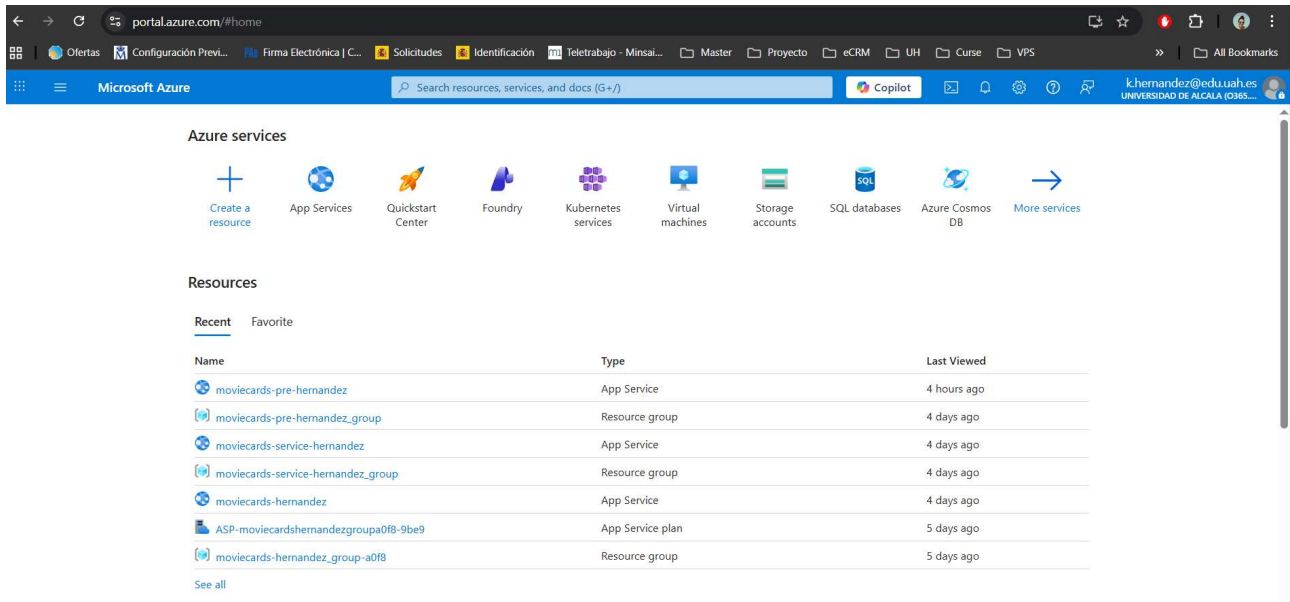
Gestión de Versiones y Control de Código

El proyecto implementó una estrategia de ramificación Git basada en Git Flow simplificado, utilizando ramas de características para el desarrollo de nuevas funcionalidades como la integración del servicio REST externo y la adición del campo deadDate. Cada funcionalidad se desarrolló en ramas independientes (modificar-aplicacion, modificar-app-deadDate), permitiendo trabajo paralelo sin afectar la rama principal. Los cambios se integraron mediante Pull Requests (#10, #16) que requerían revisión y aprobación antes del merge a master, garantizando calidad y estabilidad. Esta metodología facilitó la implementación progresiva de los requisitos del trabajo manteniendo un historial claro y desplegando automáticamente a producción tras cada integración exitosa.



Arquitectura de Despliegue en Azure

El proyecto utiliza Azure App Service con tres recursos principales desplegados: la aplicación principal de producción (moviecards-herandez.azurewebsites.net), un entorno de pre-producción (moviecards-pre-herandez.azurewebsites.net) para validación previa, y el servicio REST independiente (moviecards-service-herandez.azurewebsites.net) que proporciona la API consumida por la aplicación web, implementando una arquitectura separada entre frontend y backend con despliegue continuo gestionado mediante GitHub Actions.



Desafíos Encontrados y Soluciones

Desafío 1: Migración a Servicio Externo

Problema: Adaptar la aplicación para consumir un servicio REST externo en lugar de acceso directo a base de datos.

Solución: Implementación de RestTemplate y modificación de los servicios para realizar llamadas HTTP.

Desafío 2: Pruebas con RestTemplate

Problema: Las pruebas unitarias fallaban al mockear RestTemplate.

Solución: Uso correcto de @InjectMocks y configuración específica de los mocks para cada endpoint.

Desafío 3: Pipeline Complejo

Problema: Configurar un pipeline con múltiples stages y condiciones.

Solución: Diseño cuidadoso de dependencias entre jobs y condiciones de ejecución.

Desafío 4: Calidad del Código

Problema: Reducir problemas críticos en SonarQube por debajo de 5.

Solución: Refactorización de código, manejo adecuado de excepciones y eliminación de código duplicado.

Aprendizajes Obtenidos

- 1.**Arquitectura de microservicios:** Separación clara entre frontend y backend
- 2.**CI/CD avanzado:** Configuración de pipelines complejos con múltiples entornos
- 3.**Calidad del código:** Uso de SonarQube para mantener estándares de calidad
- 4.**Pruebas automatizadas:** Implementación completa de pruebas unitarias, de integración y end-to-end
- 5.**Despliegue en la nube:** Gestión de múltiples entornos en Azure